

Lucrare de laborator : 1

Programarea socketurilor in visual c++

1.Introducere

Limbajul C++, prin setul de biblioteci API, pune la dispoziție suport nativ pentru aplicații în rețea. Cel mai simplu mod de a realiza comunicația între două mașini este prin utilizarea [socket-urilor](#) (soclurilor) de comunicație. Socket-urile sunt mecanisme de comunicație în rețea (dezvoltate la Universitatea Berkeley din California) care pot folosi orice protocol de comunicație, deși, în general, se folosesc protocoalele din stiva TCP/IP.

Orice calculator conectat la Internet trebuie să aibă o adresă prin care să fie identificat, adresă care se numește adresă IP (Internet Protocol). O adresă IPv4 (versiunea 4) este formată din 4 octeți, iar reprezentarea ei externă se face printr-o secvență de 4 numere separate prin puncte (ex: 192.168.0.1). Pe lângă această reprezentare, se mai pot folosi nume simbolice grupate în domenii, iar corespondențele dintre adresele IP și numele simbolice sunt stocate într-o bază de date distribuită, accesată prin serviciul numelor de domenii (DNS – Domain Name Service).

Socket-urile se pot folosi într-unul din două moduri posibile:

- socluri neconectate;
- socluri conectate (orientate pe conexiune).

În cazul soclurilor neconectate, se transmit pachete în protocolul UDP ([User Datagram Protocol](#)) între procese între care nu există o legătură permanentă, iar pachetele de date (datagramele - datagrams) sunt independente între ele și conțin adresa destinației pentru a putea fi rutate.

În cazul socket-urilor conectate, se stabilește mai întâi o legătură între două socket-uri aparținând unor procese distincte (posibil plasate în stații diferite, conectate printr-o rețea de comunicație). Transmisia pe socluri conectate folosește protocolul TCP ([Transmission Control Protocol](#)) care este un protocol fiabil (cu verificări ale datelor transmise și recepționate), spre deosebire de protocolul UDP care este un protocol nefiabil.

Pentru ca mai multe aplicații să poată avea acces la rețea în același timp, pachetele pentru fiecare aplicație trebuie să fie separate. Acest lucru se face cu ajutorul [porturilor de comunicații](#). Un port este reprezentat de un număr pe 16 biți și fiecare aplicație folosește un port unic pentru a comunica în rețea. O conexiune are două porturi: cel local, alocat de sistemul de operare la inițierea conexiunii, și cel al server-ului, care este constant, în funcție de aplicație. Fiecare protocol standard are asociat un port conform unei liste menținute de cei de la [Internet Assigned Numbers Authority](#).

Ca exemple:

- port 20 - File Transfer Protocol (date)
- port 21 - File Transfer Protocol (control)
- port 22 - Secure Shell (SSH)
- port 25 - Simple Mail Transfer Protocol (SMTP)
- port 80 - Hyper Text Transfer Protocol (HTTP)
- etc.

2. Modelul Client-Server:

Majoritatea comunicațiilor între procese utilizează modelul de server client. Acești termeni se referă la cele două procese care vor comunica între ele. Unul dintre cele două procese, clientul, se conectează la celălalt proces, serverul, de obicei, pentru a face o cerere de informații. O analogie bună este o persoană care face un apel telefonic către o altă persoană.

Observați clientul trebuie să știe despre existența și adresa serverului, dar serverul nu trebuie să cunoască adresa (sau chiar existența) clientului înainte ca conexiunea să fie stabilită.

Observați, de asemenea, că, odată ce se stabilește o conexiune, ambele părți pot trimite și primi informații.

Sistemul solicită stabilirea unei conexiuni este oarecum diferit pentru client și server, dar ambele implică construcția de bază a unui socket.

Un socket este un capăt al unui canal de comunicare interproces. Cele două procese fiecare își stabilește propriul socket.

Pașii implicați în stabilirea unui socket pe partea clientului sunt după cum urmează:

1. Creați un socket cu apelul de sistem `socket()`
2. Conectați socketul la adresa serverului utilizând apelul de sistem `connect()`
3. Trimiteți și primiți date. Există mai multe moduri de a face acest lucru, dar cel mai simplu este să utilizați apelurile de sistem : `read()` și `write()`.

Pașii implicați în stabilirea unui socket pe partea serverului sunt după cum urmează:

1. Creați un socket cu apelul de sistem `socket()`
2. Legați socketul la o adresă utilizând apelul de sistem `bind()`. Pentru o socket de server de pe Internet, o adresă constă dintr-un număr de port de pe aparatul gazdă.
3. Ascultați pentru conexiuni cu apelul de ascultare `listen()`
4. Acceptați o conexiune cu apelul de sistem `accept()`. Acest apel blochează în mod obișnuit până când un client se conectează la server.
5. Trimiteți și primiți date

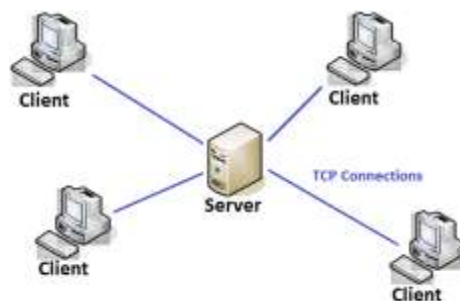


Figura 1.1: Arhitectura Client-Server

3. Tipuri de Socket-uri

Când se creează un soclu, programul trebuie să specifice domeniul de adresă și tipul socketului. Două procese pot comunica între ele numai dacă socketurile lor sunt de același tip și în același domeniu.

Există două domenii de adrese utilizate pe scară largă, domeniul unix, în care se comunică două procese care împărtășesc un sistem comun de fișiere și domeniul de Internet, în care se comunică două procese care rulează pe oricare două gazde de pe Internet. Fiecare dintre acestea are propriul format de adresă.

Adresa unui soclu din domeniul Unix este un șir de caractere care este în principiu o intrare în sistemul de fișiere.

Adresa unui soclu din domeniul Internet constă în adresa de Internet a mașinii gazdă (fiecare computer de pe Internet are o adresă unică pe 32 de biți, denumită adesea adresa IP).

În plus, fiecare soclu are nevoie de un număr de port pe acea gazdă. Numerele de port sunt numere de 16 biți nesemnate. Numerele mai mici sunt rezervate în Unix pentru servicii standard. De exemplu, numărul portului pentru serverul FTP este 21. Este important ca serviciile standard să se afle în același port pe toate computerele, astfel încât clienții să își cunoască adresele. Cu toate acestea, numerele porturilor peste 2000 sunt în general disponibile.

Există două tipuri de socket-uri utilizate pe scară largă, prize de curent și socluri de datagrame. Conectorii de flux tratează comunicațiile ca un flux continuu de caractere, în timp ce soclurile datagramei trebuie să citească toate mesajele simultan. Fiecare utilizează propriul protocol de comunicații.

Conectorii de flux folosesc TCP (Protocolul de control al transmisiei), care este un protocol fiabil, orientat spre flux, iar soclurile de datagramă folosesc UDP (Unix Datagram Protocol), care nu este fiabil și orientat spre mesaj.

Nu e suficient doar să știm ce port folosim, evident; trebuie să știm la ce computer / server la distanță ne conectăm. La fel cum aflăm adresa de domiciliu a persoanelor pe care le vizităm înainte de a ajunge în mașină, trebuie să cunoaștem adresa IP a gazdei la care ne conectăm, dacă ne conectăm și nu doar ascultăm (un program de chat are nevoie de a le putea face ambele).

O adresă IP este un număr de identificare atribuit fiecărui computer în rețea și este format din patru seturi de cifre separate prin perioade. Puteți vizualiza adresa IP executând ipconfig.exe la promptul MSDOS.

Pentru exemplele afișate în acest tutorial, vom folosi ceea ce se numește "adresa de buclă" pentru a testa programul nostru de chat fără a fi conectat la Internet. Această adresă este 127.0.0.1. Ori de câte ori încercați să faceți o conexiune la această adresă IP, computerul va trimite cererea înapoi la computer și va încerca să găsească un server pe portul specificat. În acest fel, puteți avea serverul și clientul care rulează pe același computer.

Pentru că noi, ca oameni, suntem foarte capabili să uităm lucrurile și pentru că nu am putea să sperăm să ne amintim o mulțime de numere pentru fiecare site pe care îl vizităm, niște indivizi inteligenți au venit în minunata idee de "nume de domenii". Acum, avem niște nume mai mici, cum ar fi www.yahoo.com și www.cia.gov, care reprezintă adrese IP care sunt mult mai ușor de memorat decât seturi de cifre ciudate. Când introduceți unul din aceste nume în fereastra browserului dvs., adresa IP a numelui de domeniu este privită printr-un "router" și, odată obținută (sau gazda este "rezolvată"), browserul poate contacta serverul care locuiește la acea adresă.

De exemplu, să spun că sun un operator pentru că nu-mi pot aminti numărul de telefon al prietenei mele. Deci, spun doar operatorului ce este numele său (și alte câteva detalii, dar asta

nu este important) și ea îmi dă cu plăcere cifrele. Asta se întâmplă când se face o solicitare pentru o adresă IP a oricărui nume de domeniu.

Avem două API care îndeplinesc această sarcină. Este o idee bună să vă asigurați și să verificați dacă cineva care vă folosește un program numește un nume de domeniu în locul unei adrese IP, astfel încât programul dvs. să poată căuta adresa IP corectă înainte de a continua. Majoritatea oamenilor, oricum, nu vor dori să-și amintească nici o adresă IP, deci cel mai probabil va trebui să traduceți numele de domeniu în adrese IP înainte de a putea stabili o conexiune - ceea ce presupune conectarea calculatorului la Internet. Apoi, odată ce ai adresa, ești pregătit să te conectezi.

4. Ordinea octetilor

Chiar când ați crezut că toate aceste chestii vor fi simple și ușoare, trebuie să începem să discutăm ordinea bitilor. Acest lucru se datorează faptului că computerele Intel și protocoalele de rețea utilizează comenzi de octeți invers unul de celălalt și trebuie să acoperim fiecare port și adresa IP în ordine de byte în rețea înainte de a le trimite; altfel vom avea o mare amestecare. Portul 25, atunci când nu este inversat, nu va ajunge la portul 25. Deci, trebuie să ne asigurăm că vorbim aceeași limbă ca și serverul când încercăm să comunicăm cu el.

Din fericire, nu trebuie să codificăm manual toate funcțiile de conversie; deoarece Microsoft ne oferă cu amabilitate câteva API pentru a face acest lucru, de asemenea. Cele patru funcții care sunt utilizate pentru a schimba ordinea byte a unui număr de IP sau port sunt următoarele:

De exemplu, înainte de a specifica portul pe care urmează să-l ascultați sau pe care îl vom conecta, va trebui să folosim funcția `htons()` pentru a converti numărul în ordinea byte-ului de rețea. Rețineți că după ce ați utilizat `inet_addr()` pentru a converti o adresă IP șir la formularul cerut, vom fi returnați adresa în ordinea corectă a rețelei, eliminând necesitatea de a evoca `htonl()`. O modalitate ușoară de a face diferența între `htons()` și `htonl()` este de a considera numărul de port ca fiind cel mai scurt, iar IP ca număr mai lung (ceea ce este adevărat - o adresă IP constă din patru seturi de până la trei cifre separate în funcție de perioade, față de un singur număr de port).

5. Initializarea unui socket

Toate procesele (aplicații sau DLL-uri) care apelează funcțiile Winsock trebuie să inițializeze utilizarea DLL-urilor Socket-urilor Windows înainte de a efectua alte apeluri ale funcțiilor Winsock. Acest lucru asigură, de asemenea, că Winsock este suportat pe sistem.

Când creați un soclu, va trebui să treceți "familia de adrese", tipul "socket" și "tipul de protocol". Cu excepția cazului în care faceți o lucrare specială (sau ciudată), care depășește sfera acestui raport, trebuie de obicei să treci `AF_INET` ca familie de adrese implicită. Acest parametru specifică modul în care vor fi interpretate adresele calculatorului.

Există mai mult decât un singur tip de soclu; de fapt, sunt multe altele. Trei dintre cele mai obișnuite includ: socketul goale, streamuri și socluri de date. Adaptoarele de flux sunt însă cele pe care le folosim în acest tutorial, deoarece avem de-a face cu protocoalele TCP, așa că vom specifica `SOCK_STREAM` drept al doilea parametru pentru socket (`()`).

Va trebui să completați informații despre gazda la distanță la care vă conectați și apoi să transmiteți un pointer acestei structuri funcției magice `connect()`. Această structură și API sunt enumerate mai jos. Rețineți că parametrul `sin_zero` nu este necesar și este lăsat necompletat.

```

struct sockaddr_in
{
    short    sin_family;
    u_short  sin_port;
    struct    in_addr sin_addr;
    char     sin_zero[8];
};

int PASCAL connect(SOCKET, const struct sockaddr*, int);

```

Fig 1.2 :Structura unui socket.

```

#include <winsock.h>
SOCKET s; //Initializarea socketului
//CONNECTTOHOST - Se conectează la o gazdă la distanță
bool ConnectToHost(int PortNo, char* IPAddress)
{
    //Start Winsock...
    WSADATA wsadata;
    int error = WSASStartup(0x0202, &wsadata);
    //Daca s-a intamplat ceva ?
    if (error)
        return false;
    //Am obținut versiunea Winsock potrivită?
    If (wsadata.wVersion != 0x0202)
    {
        WSACleanup(); //Curata Winsock
        return false;
    }
    //Completați informațiile necesare inițializării unui soclu ..
    SOCKADDR_IN target; //Socket cu informatii de adresa
    target.sin_family = AF_INET; // adresa familiei in Internet
    target.sin_port = htons (PortNo); //Portul la care ne conectam
    target.sin_addr.s_addr = inet_addr (IPAddress); //IP tinta
    s = socket (AF_INET, SOCK_STREAM, IPPROTO_TCP); //Creare socket
    if (s == INVALID_SOCKET)
    {
        return false; //Nu s-a putut creea socket
    }
    //Incearca conectarea...
    if (connect(s, (SOCKADDR *)&target, sizeof(target)) == SOCKET_ERROR)
    {
        return false; //Nu s-a putut conecta
    }
    else
        return true; //Success
}

//CLOSECONNECTION - închide soclul și închide orice conexiune de pe el
void CloseConnection ()
{
    //Închideți soclul dacă există
    if (s)
        closesocket(s);

    WSACleanup(); //Curata Winshok
}

```

Figura 1.3 : Cod pentru conectarea la un host

6. Așteptarea conexiunilor:

Când acționați ca server, puteți primi cereri pentru o conexiune pe portul pe care îl ascultați: de exemplu, un computer la distanță dorește să vorbească cu computerul, va întreba mai întâi serverul dacă dorește sau nu să stabilească o conexiune. Pentru ca o conexiune să fie făcută, serverul dvs. trebuie să accepte () solicitarea de conectare. Rețineți că "serverul" decide dacă să stabilească conexiunea sau nu. În cele din urmă, ambele computere sunt conectate și pot schimba date.

Deși funcția listen () este cel mai simplu mod de a asculta pe un port și de a acționa ca server, nu este cel mai de dorit. Veți afla rapid când veți încerca să înghețe programul dvs. până când se va efectua o conexiune de intrare, deoarece funcția de ascultare () este o funcție de "blocare" - poate efectua o singură sarcină la un moment dat și nu se va întoarce până când nu este conectată o conexiune în așteptare.

Aceasta este cu siguranță o problemă, dar există câteva soluții pentru aceasta. În primul rând, dacă sunteți familiarizat cu aplicațiile cu mai multe filete (rețineți că nu vorbim despre firele TCP aici), atunci puteți plasa codul serverului pe un fir separat care, atunci când a început, nu va îngheța întregul program și eficiența programul părinte nu va fi astfel împiedicat. Aceasta este într-adevăr mai mult de o durere pe care trebuie să fie; deoarece ați putea înlocui funcția listen () cu prize "asincrone". Dacă ți-am atras atenția cu numele important, poți sări la următoarea secțiune dacă vrei, dar îți recomand să rămâi cu mine aici și să înveți fundamentele. Ne vom spiff codul mai târziu; dar pentru moment, să ne concentrăm pe lucrurile esențiale.

Înainte de a vă gândi chiar să ascultați pe un port, trebuie:

- Inițializați Winsock
- Porniți un soclu și asigurați-vă că acesta returnează o valoare nonzero, ceea ce înseamnă succes
- Completați structura SOCKADDR_IN cu datele necesare, inclusiv familia de adrese, portul și adresa IP.
- Utilizați bind () pentru a lega soclul la o adresă IP specifică (dacă ați specificat inet_addr ("0.0.0.0") sau htonl (INADDR_ANY) ca secțiunea sin_addr a SOCKADDR_IN, puteți lega la orice adresă IP)

Primul parametru de ascultare () trebuie să fie la un soclu pe care l-ați inițializat anterior. Desigur, orice port pe care este atașat acest soclu este portul pe care îl veți asculta. Apoi, puteți specifica, împreună cu parametrul următor și ultim, câte computere la distanță pot comunica cu serverul dvs. în același timp. În general, cu toate acestea, dacă nu doriți să excludem toate conexiunile, dar doar una sau mai multe, doar trecem SOMAXCONN (SOcket MAX CONNecTion) ca parametru final pentru a asculta (). Dacă socketul este în sus și funcționează bine, totul ar trebui să meargă bine, iar atunci când o solicitare de conexiune a fost recepționată, ascultarea () va reveni. Acesta este indiciul dvs. pentru a apela accepta (), dacă doriți să stabiliți o conexiune.

```

#include <windows.h>
#include <winsock.h>
SOCKET s;
WSADATA w;
//LISTENONPORT - Asculta pe un port specificat pentru conexiuni sau date primite
int ListenOnPort(int portno)
{
    int error = WSStartup (0x0202, &w);    // Fill in WSA info
    if (error)
    {
        return false; //Din anumite motive, nu am putut să începem Winsock
    }
    if (w.wVersion != 0x0202) //Versiunea Winsock greșită?
    {
        WSACleanup ();
        return false;
    }
    SOCKADDR_IN addr; // Structura adresei pentru un soclu TCP
    addr.sin_family = AF_INET;    // Address de familie
    addr.sin_port = htons (portno);    // Asigneaza port la acest socket
    //Acceptați o conexiune de la orice IP utilizând INADDR_ANY
    addr.sin_addr.s_addr = htonl (INADDR_ANY);
    s = socket (AF_INET, SOCK_STREAM, IPPROTO_TCP); // Create socket
    if (s == INVALID_SOCKET)
    {
        return false; //Nu continuați dacă nu am putut crea un // soclu!
    }
    if (bind(s, (LPSOCKADDR)&addr, sizeof(addr)) == SOCKET_ERROR)
    {
        //Nu am putut lega (acest lucru se va întâmpla dacă încercați să vă legați la același lucru
        //socket more than once)
        return false;
    }
    listen(s, SOMAXCONN);
    //Nu uitați să curățați cu CloseConnection ()!
}

```

Figura 1.4 : Cod partea de server

7. Socketuri asincrone

C++ ne dă un avantaj aici că majoritatea limbajelor de programare la nivel înalt nu au; și anume, deoarece nu trebuie să mergem la lungimea suplimentară a "sub-clasării" ferestrei parinte înainte de a putea folosi soclurile asincrone. E deja făcut pentru noi, deci tot ce trebuie să faceți este să adăugați codul de manipulare în dispozitivul de gestionare a mesajelor. Acest lucru se datorează faptului că prizele asincrone, după cum veți vedea, depind de posibilitatea de a trimite mesajele de program când se face o solicitare de conexiune, se primesc date etc.

Acest lucru îi permite să aștepte în tăcere în fundal, fără a deranja programul dvs. părinte sau împiedicarea productivității, deoarece comunică doar atunci când este necesar. Există un preț relativ mic de plătit, de asemenea, pentru că într-adevăr nu necesită multă codificare suplimentară. Înțelegerea modului în care funcționează toate acestea ar putea dura puțin, dar cu siguranță veți fi mulțumit că ați avut timp să înțelegeți prizele asincrone. Va economisi multe probleme pe termen lung. În loc să trebuiască să refacem și să modificăm tot codul pe care l-am scris până acum, făcând un soclu asincron pur și simplu necesită o linie suplimentară de cod după funcția listen (). Desigur, managerul de mesaje trebuie să fie pregătit să accepte următoarele mesaje:

8. Exemplu Aplicatie:

Se va crea o aplicatie client server in visual c++ utilizand mediul de dezvoltare visual studio urmand pasii detalati mai sus, aplicatia server va afisa un mesaj de confirmare la fiecare client conectat si “mesajul zilei”.

```
#define _WINSOCK_DEPRECATED_NO_WARNINGS
#pragma comment(lib, "ws2_32.lib")
#include <WinSock2.h>
#include <iostream>
int main()
{
    //Winsock pornire
    WSADATA wsaData;
    WORD DllVersion = MAKEWORD(2, 1);
    if (WSAStartup(DllVersion, &wsaData) != 0) //Dacă WSAStartup returnează altceva decât 0, înseamnă că a
    { //apărut o eroare în Startup WinSock.
        MessageBoxA(NULL, "Pornirea Winsock a esuat", "Error", MB_OK | MB_ICONERROR);
        exit(1);
    }
    SOCKADDR_IN addr; //Adresă care trebuie legată la soclul de conectare
    int sizeofaddr = sizeof(addr); //Aveți nevoie de sizeofaddr pentru funcția de conectare
    addr.sin_addr.s_addr = inet_addr("127.0.0.1"); //Address = localhost (acest pc)
    addr.sin_port = htons(1111); //Port = 1111
    addr.sin_family = AF_INET; //IPv4 Socket
    SOCKET Connection = socket(AF_INET, SOCK_STREAM, NULL); //Setați soclul de conectare
    if (connect(Connection, (SOCKADDR*)&addr, sizeofaddr) != 0) //Dacă nu ne putem conecta ...
    {
        MessageBoxA(NULL, "Conexiunea a esuat", "Error", MB_OK | MB_ICONERROR);
        return 0; //Conexiunea a esuat
    }
    std::cout << "Conectat!" << std::endl;
    char MOTD[256];
    recv(Connection, MOTD, sizeof(MOTD), NULL); //Primiți mesajul din buffer în matricea MOTD
    std::cout << "MOTD:" << MOTD << std::endl;
    while (true)
    {
        Sleep(10);
    }
}
```

Figura 1.5: Aplicatia Client

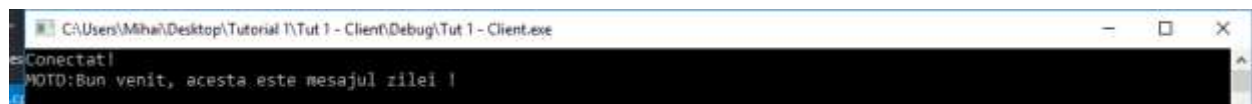


Figura 1.6: Aplicatia Client cand se conecteaza la server

```

#define _WINSOCK_DEPRECATED_NO_WARNINGS
#pragma comment(lib, "ws2_32.lib")
#include <WinSock2.h>
#include <iostream>
int main()
{
    //WinSock pornire
    WSADATA wsaData;
    WORD DllVersion = MAKEWORD(2, 1);
    if (WSAStartup(DllVersion, &wsaData) != 0) //Dacă WSAStartup returnează altceva decât 0, înseamnă că a apărut o eroare în Startup WinSock.
    {
        MessageBoxA(NULL, "WinSock a esuat", "Error", MB_OK | MB_ICONERROR);
        return 0;
    }

    SOCKADDR_IN addr; //Adresa pe care o vom lega la soclul de ascultare
    int addrlen = sizeof(addr); //lungimea adresei(necesară pentru a accepta apelul)
    addr.sin_addr.s_addr = inet_addr("127.0.0.1"); //Broadcast local
    addr.sin_port = htons(1111); //Port
    addr.sin_family = AF_INET; //IPv4 Socket
    SOCKET sListen = socket(AF_INET, SOCK_STREAM, NULL); //Creează un soclu pentru a asculta conexiuni noi
    bind(sListen, (SOCKADDR*)&addr, sizeof(addr)); //Legată adresa la soclu
    listen(sListen, SOMAXCONN); //Locațiile sListen socket într-o stare în care ascultă o conexiune primită. Notă: SOMAXCONN = Conexiuni maxime
    SOCKET newConnection; //Socket pentru a menține conexiunea
    newConnection = accept(sListen, (SOCKADDR*)&addr, &addrlen); //Acceptarea unei noi conexiuni
    if (newConnection == 0) //Dacă pica conexiunea cu clientul
    {
        std::cout << "Nu s-a putut accepta conexiunea clientului." << std::endl;
    }
    else //daca conexiunea cu clientul este acceptata
    {
        std::cout << "Client conectat!" << std::endl;
        char MOTO[256] = "Bun venit, acesta este mesajul zilei ! "; //Creează un buffer cu mesajul zilei
        send(newConnection, MOTO, sizeof(MOTO), NULL); //Trimite MOTO buffer
    }
    system("pause");
    return 0;
}

```

Figura 1.7: Aplicatia Server

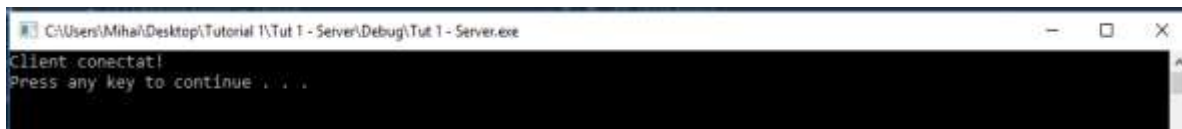


Figura 1.8: Aplicatia Server dupa ce se conecteaza un client

9. Aplicatii de implementat:

1. Implementati aplicatiile de mai sus.
2. Modificati aplicatia de la punctul 8 pentru a afisa 2 mesaje in momentul in care se apasa o tasta.
3. Folosind threaduri faceti aplicatia de mai sus sa accepte conexiuni paralele cu mai multi client, fiecare client cu thread-ul lui.

Bibliografie :

1. http://www.linuxhowtos.org/C_C++/socket.htm
2. <https://www.codeproject.com/Tags/C--plus----plus-->
3. https://en.wikipedia.org/wiki/Network_socket
4. [https://msdn.microsoft.com/en-us/library/windows/desktop/bb530741\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/bb530741(v=vs.85).aspx)
5. <https://www.binarytides.com/winsock-socket-programming-tutorial>